Computer Networks
Project Report

on

EVNet Sentinel

Network Intrusion Detection for
Electric Vehicle Charging Infrastructure

Subject: Computer Networks

Academic Year: 2026–2027

Prepared by: Group 2

Roll No.	Name	Role
24102A0013	Mukta Varak	Cloud Deployment & DevOps
24102A0018	Shruti Chaurasiya	Data Visualization & Testing
24102A0021	Shardul Chogale	ML Pipelining & Evaluation
24102A0022	Neha Chavhan	Frontend & API Integration

Guide: Prof. Mahesh Khandke
Department of Computer Engineering

Vidyalankar Institute of Technology, Mumbai

October 2026

Contents

1 Introduction **1**

2 EVCS Network Architecture & Protocols **1**
2.1 Communication Protocol Stack 1
2.2 Network Monitoring Point 2

3 CICEVSE2024: Network Traffic Dataset **2**
3.1 Dataset Composition 2
3.2 Key Network Flow Features 3

4 Network Attack Taxonomy in EVCS **4**
4.1 Denial-of-Service (DoS / DDoS) 4
4.2 Reconnaissance (Network Scanning) 4
4.3 Brute-Force & Application-Layer Attacks 4

5 Data Leakage Audit: Network-Level Findings **5**

6 Corrected Intrusion Detection Methodology **6**
6.1 Leak-Free Preprocessing Pipeline 6
6.2 Model Benchmarking 6

7 Closed-Loop Network Response System **7**

8 System Architecture — Network Perspective **8**

9 Conclusion **9**

References **9**

1. Introduction

The rapid adoption of Electric Vehicles (EVs) has led to a parallel expansion of Electric Vehicle Charging Station (EVCS) networks. These stations are no longer standalone devices; they are networked endpoints communicating over TCP/IP with Charging Station Management Systems (CSMS), energy grid operators, and billing platforms. This connectivity, while enabling features like remote diagnostics, dynamic pricing, and vehicle-to-grid (V2G) energy trading, also exposes the infrastructure to conventional network-based cyberattacks.

Core Research Question

Makhmudov et al. (2025) reported **98.4%** accuracy for an online intrusion detector (Adaptive Random Forest + ADWIN) on the CICEVSE2024 network-traffic dataset. *Does this number genuinely measure attack detection, or is it an artefact of data leakage?*

EVNet Sentinel is a network-level Intrusion Detection System (IDS) that audits the above claim, identifies five distinct sources of data leakage, and rebuilds the detection pipeline honestly. This report examines the project through the lens of **Computer Networks**, focusing on the network protocols, traffic characteristics, attack taxonomy, and flow-level feature engineering that underpin the system.

2. EVCS Network Architecture & Protocols

2.1 Communication Protocol Stack

An EVCS network operates over a layered protocol stack that extends the standard TCP/IP model:

Table 1: EVCS Communication Protocol Stack

Layer	Standard Protocol	EVCS-Specific Protocol
Application	HTTP/JSON, WebSocket	OCPP 1.6/2.0.1
Presentation	TLS 1.2/1.3	ISO 15118 (V2G)
Transport	TCP, UDP	—
Network	IPv4/IPv6	—
Data Link	Ethernet, Wi-Fi	PLC (HomePlug Green PHY)
Physical	Cat5e/6, 802.11	Control Pilot (IEC 61851)

- **OCPP (Open Charge Point Protocol):** A WebSocket/JSON-based application protocol enabling charge point ↔ CSMS communication (boot notifications, start/stop transactions, firmware updates, and remote triggers).
- **ISO 15118 (V2G):** Governs vehicle-to-grid communication for Plug-and-Charge authentication, bidirectional energy negotiation, and certificate-based mutual TLS.
- **MQTT / AMQP:** Some deployments use lightweight messaging for telemetry aggregation from station clusters to cloud backends.

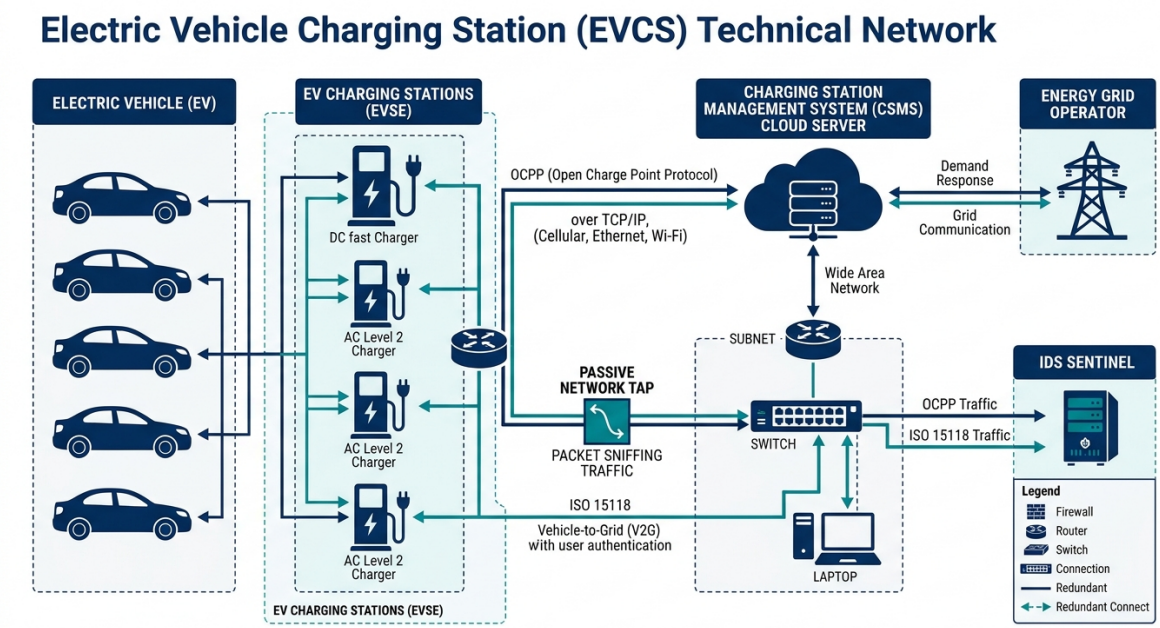


Figure 1: EVCS technical network architecture showing chargers (EVSE), CSMS cloud server, power grid communication, and the passive network tap monitoring point where flow data is captured.

2.2 Network Monitoring Point

The CICEVSE2024 dataset was generated by placing a passive network tap at the aggregation switch connecting multiple EVSE units to the CSMS. All traffic—benign transactions, firmware exchanges, and injected attack flows—was captured as full PCAP files and subsequently converted to *flow-level* records using CICFlowMeter, yielding 83 bidirectional features per flow.

3. CICEVSE2024: Network Traffic Dataset

3.1 Dataset Composition

The CICEVSE2024 dataset, published by the Canadian Institute for Cybersecurity (University of New Brunswick), contains ∼1.92 million bidirectional network flows across 15 classes (1 Benign + 14 attack types).

Table 2: CICEVSE2024 Traffic Class Distribution

Traffic Class	**Flows**	**Category**
Benign	209,987	Normal
DoS SlowHTTPTest	415,821	Denial of Service
DoS Slowloris	202,461	Denial of Service
DoS GoldenEye	58,391	Denial of Service
DoS Hulk	130,207	Denial of Service
DDoS LOIC-HTTP	73,108	Distributed DoS
DDoS HOIC	56,992	Distributed DoS
Nmap OS Scan	48,721	Reconnaissance
Nmap TCP Scan	89,134	Reconnaissance
Nmap UDP Scan	61,222	Reconnaissance
+ 4 additional attack classes (FTP Patator, SSH Patator, etc.)		
Total	**∼1.92M**	**15 classes**

3.2 Key Network Flow Features

Each flow record contains 83 features extracted by CICFlowMeter. The most discriminative (after leakage removal) include:

Table 3: Critical Network Flow Features for Intrusion Detection

Feature	Network Significance
`dst_port`	Destination port; directly identifies port scan targets (dropped by the reference pipeline—a critical error)
`bidirectional_packets`	Total packets in the flow; high counts signal flood attacks
`bidirectional_bytes`	Total bytes; volumetric signature of DoS
`bidirectional_duration_ms`	Flow duration; Slowloris keeps connections alive far longer than benign OCPP
`src2dst_mean_ps`	Mean packet size source→destination; large payloads vs. probing SYN packets
`dst2src_packets`	Reverse-direction packets; absent in half-open scans, high in established connections
`src_port`	Ephemeral source port; bands per capture session (partial fingerprint)

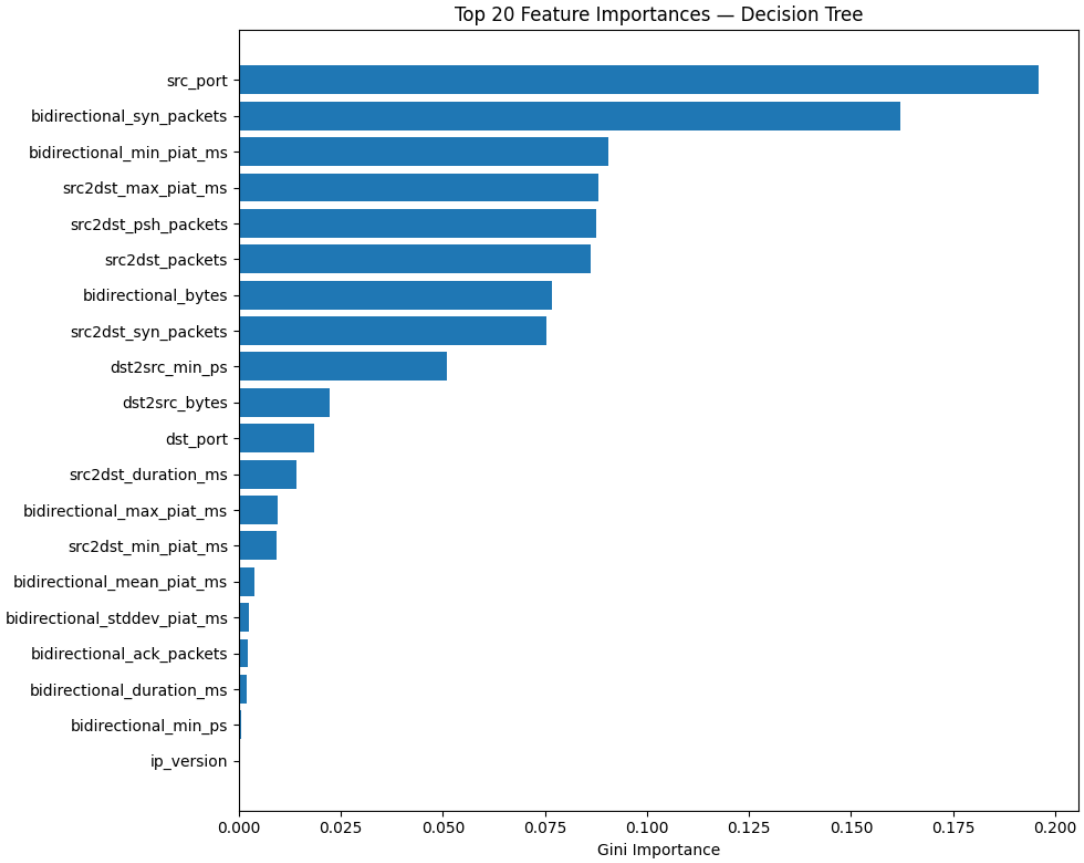


Figure 2: Top discriminative network flow features from the corrected leak-free model after removing timestamp epoch identifiers.

4. Network Attack Taxonomy in EVCS

The attacks present in CICEVSE2024 map directly to well-known network attack categories:

4.1 Denial-of-Service (DoS / DDoS)

- **Volumetric Floods** (Hulk, GoldenEye, LOIC-HTTP, HOIC): Overwhelm the EVCS with high packet/byte rates, disrupting charging session management and OCPP communication.
- **Slow-rate Attacks** (Slowloris, SlowHTTPTest): Exhaust server connection pools by holding HTTP connections open with minimal traffic—difficult to detect by volume alone; identifiable by anomalous `bidirectional_duration_ms`.

Detection Result — DoS

All flood-type attacks achieve **F1 $\geq$ 0.999** with the corrected pipeline. Slow-rate attacks are also well-detected due to distinctive flow-duration signatures.

4.2 Reconnaissance (Network Scanning)

- **Nmap TCP/UDP/OS Scans:** Probe open ports and services on EVCS endpoints. These generate many short-lived flows with varied `dst_port` values.
- **The `dst_port` problem:** The reference pipeline *drops* `dst_port`—the single feature that *defines* a port scan—making scan classification nearly impossible from remaining features alone.

Detection Result — Reconnaissance

Scan-type attacks achieve only **F1 0.10–0.27** after leakage correction. Mutual information analysis shows the best remaining feature explains only **5%** of scan-type variance—the detection ceiling is a *data limitation*, not a model limitation.

4.3 Brute-Force & Application-Layer Attacks

FTP Patator and SSH Patator attacks target authentication services on the EVCS management interface. These produce many failed-login flows with characteristic packet sizes and connection-reset patterns.

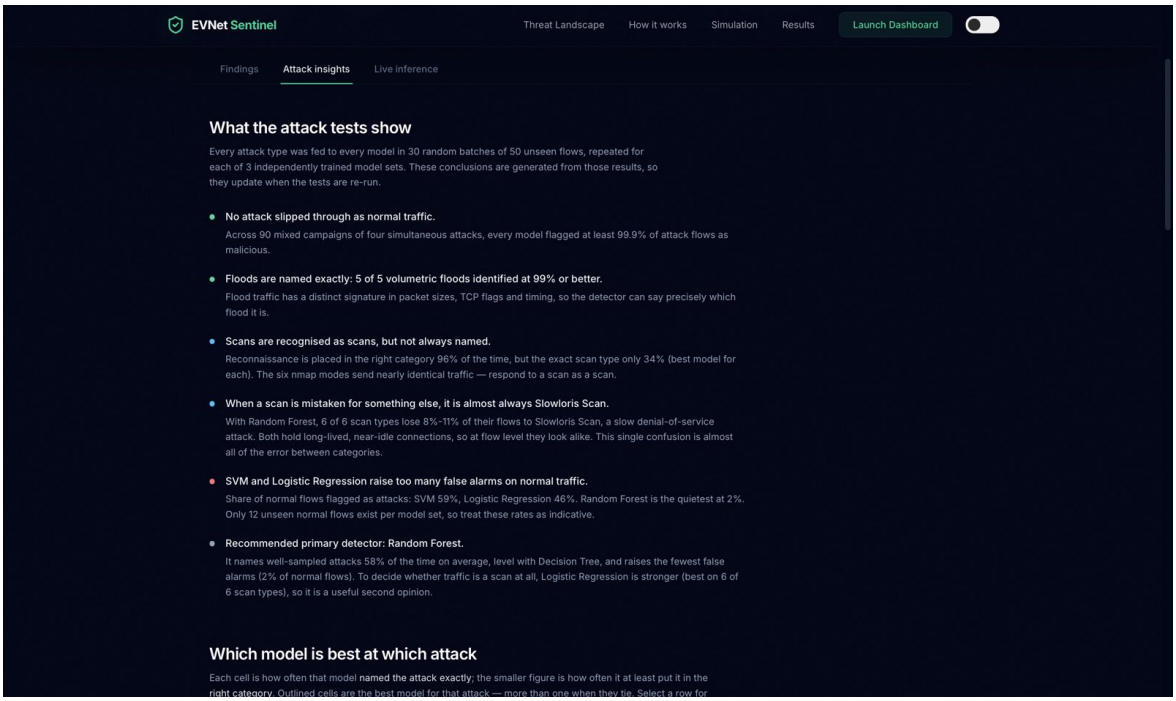


Figure 3: The "Attack Insights" console in EVNet Sentinel showing per-attack empirical performance, false alarm rates, and operator response instructions.

5. Data Leakage Audit: Network-Level Findings

The central contribution of EVNet Sentinel is identifying that the published 98.4% accuracy was inflated by non-network-behavioral features leaking capture metadata. All five findings have a network-level explanation:

Finding 1 — Timestamp Leak

Each attack type was recorded in a separate PCAP file at a distinct wall-clock time. The six `*_first_seen_ms` / `*_last_seen_ms` columns encode *when the capture was taken*, not any property of the network flow itself. A single Decision Tree on `bidirectional_first_seen_ms` alone scores **0.9976** accuracy—above the published 0.9840.

Finding 2 — `dst_port` Dropped

The reference pipeline removes `dst_port` (the destination port number), the feature that *defines* a port scan at the network level, while retaining the leaky timestamps.

Finding 3 — Stream Ordering

In original capture order, the attack label changes only 54 times across 1.92M flows. A trivial "repeat the last label" rule scores **0.99997**—any sequential model (including ARF + ADWIN) benefits from this temporal autocorrelation, not from learning network patterns.

Finding 4 — Drift Alarms = File Seams

42 of 43 ADWIN drift alarms fall within 100 flows of a join between PCAP files ($p < 0.0005$). The drift detector responded to *file concatenation boundaries*, not genuine concept drift in network traffic.

Finding 5 — `src_port` Fingerprint

Ephemeral source ports band per capture session. `src_port` acts as a partial session identifier rather than a behavioral network feature. Removing it costs Decision Tree twice the performance it costs Random Forest.

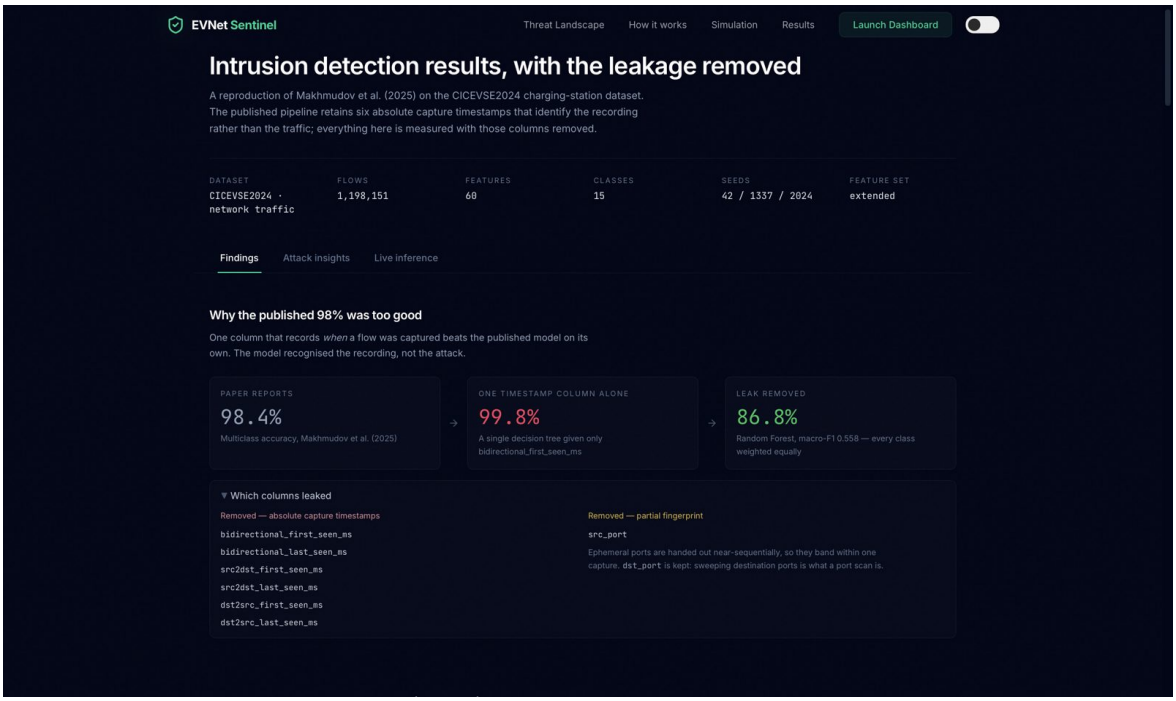


Figure 4: The "Findings" dashboard highlighting the audit evidence: timestamp-only classification (99.76%), capture-order autocorrelation (99.99%), and true corrected macro-F1 (0.558).

6. Corrected Intrusion Detection Methodology

6.1 Leak-Free Preprocessing Pipeline

The corrected pipeline applies the following network-aware transformations:

1. **Drop timestamp columns:** All six `*_first_seen_ms` / `*_last_seen_ms` epoch columns removed.
2. **Retain `dst_port`:** The destination port is a legitimate network feature essential for scan detection.
3. **Shuffle before split:** Destroy capture-order autocorrelation; perform stratified 80/20 train/test split.
4. **Deduplicate:** Remove exact-duplicate flow rows (an artefact of CICFlowMeter window overlap).
5. **StandardScaler:** Fit on training set only to prevent test-set information leakage.

6.2 Model Benchmarking

Six classifiers were evaluated on the corrected, leak-free dataset across 3 random seeds:

Table 4: Corrected Model Benchmark Results (Leak-Free, 3-Seed Average)

Model	Macro-F1	Accuracy	False Alarms	Notes
Random Forest	**0.558 ± 0.008**	0.868	3%	Highest macro-F1
Decision Tree	0.557 ± 0.010	0.868	11%	
XGBoost	0.551 ± 0.011	0.869	2%	**Recommended** (fewest false alarms)
LightGBM	0.550 ± 0.004	0.868	3%	
Logistic Regression	0.436 ± 0.017	0.863	44%	Best on scans
SVM	0.411 ± 0.017	0.862	58%	

Key observation: All four tree-based models are statistically tied at ∼0.55 macro-F1. The accuracy of ∼0.87 is driven by the dominant flood classes; **macro-F1** is the correct headline metric because it equally weights all 15 classes, including the poorly-detected scans.

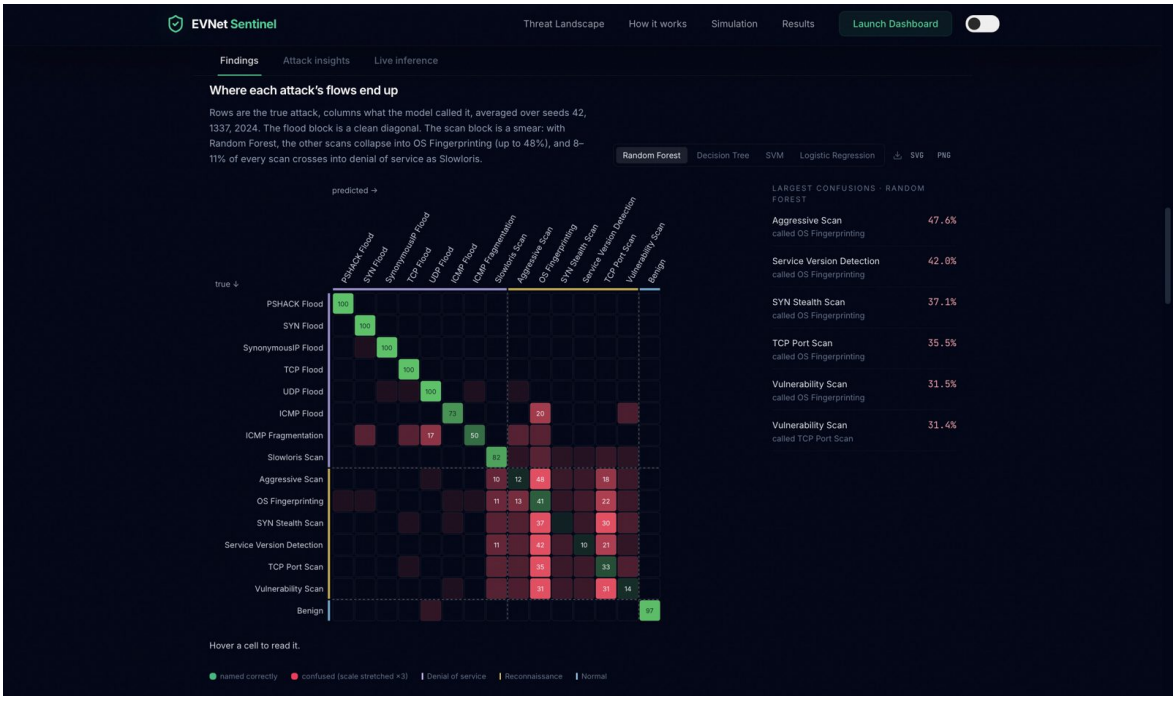


Figure 5: Interactive multi-class confusion matrix explorer showing sharp diagonal concentration for volumetric floods alongside horizontal dispersion across scan variants.

7. Closed-Loop Network Response System

Beyond detection, EVNet Sentinel implements a closed-loop containment policy modeled on real network defense practices:

Table 5: Threat Level Escalation and Network Response Actions

Threat Level	Trigger	Network Response
Safe	No malicious flows	Normal EVCS operation
Elevated	Scan detected	Restrict scanning IP; rate-limit connections
Not Safe	Flood / DDoS detected	Quarantine attacker IP at firewall; terminate malicious sessions

The response system was replayed over all 20 recorded attacker–attack pairs in the dataset. A scanner that later escalates to flooding is automatically quarantined on its first flood flow—demonstrating

adaptive, stateful network defense.

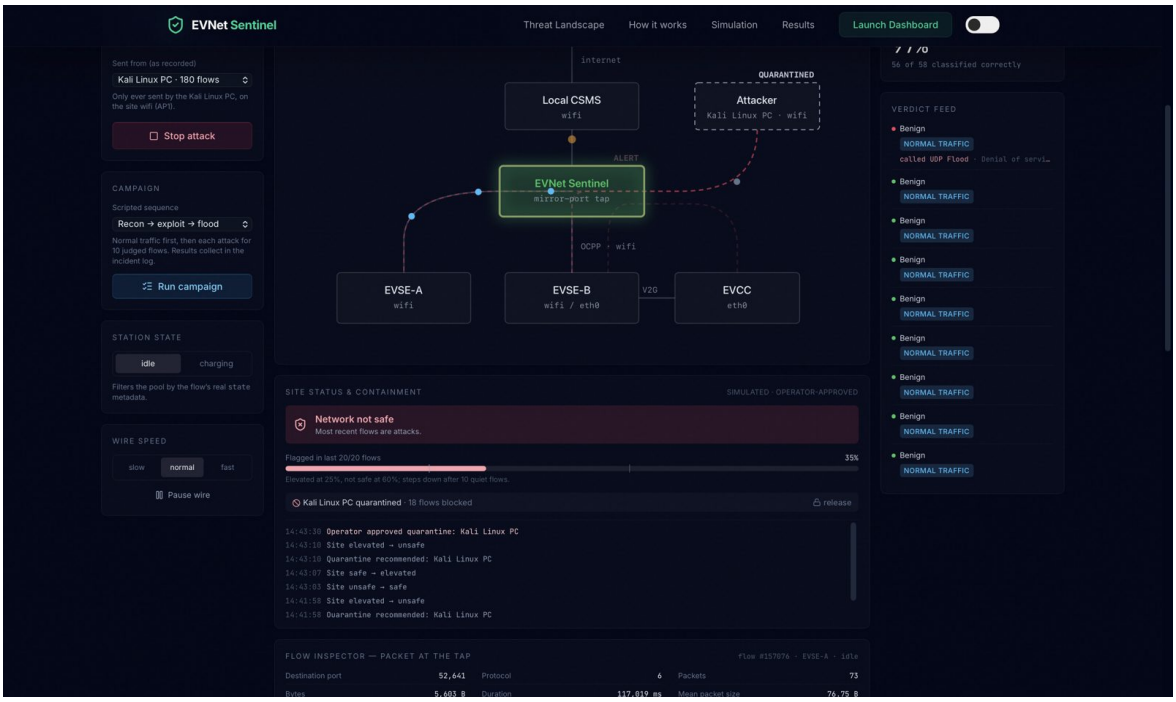


Figure 6: Interactive simulation console demonstrating state transition to "Network Not Safe" with closed-loop operator containment quarantine triggered on malicious traffic.

8. System Architecture — Network Perspective

The EVNet Sentinel deployment uses a decoupled client-server architecture:

Network Communication

- **FastAPI Backend** (`:8000`): Serves REST endpoints (`POST /predict`) for single-flow classification. Loads trained models from `saved_models/` at startup via a dynamic model registry.
- **Next.js Frontend** (`:3100`): Reads pre-exported JSON results from `frontend/data/` — no runtime API dependency, enabling deployment on a fresh clone without the dataset.
- **WebSocket Channel**: Real-time push of predictions and threat-level changes from backend to dashboard.
- **Docker Containerization**: Single-stage build preserving Rust/C++ compilation artefacts required by the `river` streaming ML library.

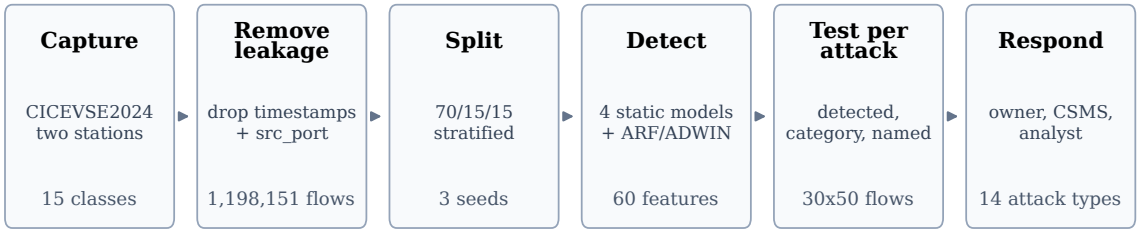


Figure 7: Comprehensive end-to-end system and ML pipeline architecture spanning CICEVSE2024 ingestion, leak-free training, dynamic FastAPI model registry, and Next.js presentation.

9. Conclusion

This project demonstrates the critical importance of **network-aware feature engineering** in building reliable intrusion detection systems. The five leakage findings show that high-accuracy claims can arise from capture metadata rather than genuine network behavior. After correction:

- **Flood attacks** (DoS/DDoS) are near-perfectly detectable (F1 ≥ 0.999) from volumetric flow features.
- **Reconnaissance scans** remain a hard problem (F1 0.10–0.27) due to data-level limitations—the single best feature explains only 5% of scan-type variance.
- The **corrected macro-F1 of 0.558 $\pm$ 0.008** is the honest baseline for future EVCS network security research on this dataset.
- A **closed-loop response system** demonstrates how detection feeds into practical network containment (quarantine, restriction, session termination).

Future work includes deploying the IDS on live OCPP traffic streams, incorporating deep-packet inspection for encrypted V2G payloads, and investigating graph-neural-network approaches for modeling station-to-station communication patterns.

References

[1] F. Makhmudov, D. Kilichev, U. Giyosov, F. Akhmedov, "Online Machine Learning for Intrusion Detection in Electric Vehicle Charging Systems," *Mathematics*, vol. 13, no. 5, p. 712, 2025.

[2] E. D. Buedi, A. A. Ghorbani, S. Dadkhah, R. L. Ferreira, "Enhancing EV Charging Station Security Using a Multi-dimensional Dataset: CICEVSE2024," in *Data and Applications Security and Privacy XXXVIII (DBSec 2024)*, LNCS vol. 14901, Springer, 2024, pp. 171–190.

[3] C. Joglekar, C. Eze, D. Xiang, A. Monti, "A Hybrid Intrusion Detection System for Electric Vehicle Charging Infrastructure," *arXiv:2606.23236*, 2026.

[4] J. Bean, D. M. Manias, "Cybersecurity of Electric Vehicle Charging Infrastructure: Recent Advances, Open Challenges, and Future Directions," *arXiv:2605.24190*, 2026.

[5] A. Dridi, C. Gamboa, M. Elrawy, *et al.*, "A Survey on Cybersecurity of Autonomous and Connected Electric Vehicles," *IEEE Access*, vol. 12, pp. 1–24, 2024.

[6] Open Charge Alliance, "OCPP 2.0.1 Specification," https://www.openchargealliance.org/, 2022.

[7] ISO, "ISO 15118:2019 — Road vehicles — Vehicle to grid communication interface," International Organization for Standardization, 2019.

[8] TATU-hacker, "Intrusion Detection on Electric Vehicle Charging Systems," GitHub Repository, https://github.com/TATU-hacker/Intrusion_Detection_on_Electric_Vehicle_Charging_Systems.